IKB42603 Lab 0 - Environment Setup

Name: Arif
Course: IKB42603 Cloud Computing
Guide followed: IKB42603_Lab0_Environment_Setup_Cheatsheet.pdf

Objective

Set up and validate a local cloud-computing development environment with Docker, AWS CLI v2, Kubernetes tools (kind and kubectl), LocalStack, and supporting security tools.

Learning Outcomes

Install and verify Docker container support.
Install AWS CLI v2 and configure test credentials for LocalStack.
Install and use kind and kubectl to create a local Kubernetes cluster.
Run LocalStack as a Docker container and connect to it through the AWS CLI.
Use OpenSSL and Trivy as supporting cloud and container-security tools.

Environment

Host environment: Kali Linux terminal.
Container runtime: Docker.
Cloud CLI: AWS CLI v2.
Kubernetes tools: kind v0.23.0 and kubectl v1.33.4.
Local AWS emulator: LocalStack at http://localhost:4566.
Kubernetes cluster: ccse, with context kind-ccse.
Helper tools: OpenSSL and Trivy.

Step-by-Step Implementation

1. Install and verify Docker

Docker was installed in the Kali Linux environment. The hello-world image was run to confirm that the Docker client could contact the daemon, pull an image, create a container, and display its output.

```
docker run hello-world
```

The command displayed "Hello from Docker!", confirming that Docker was working correctly.

2. Install AWS CLI v2

The AWS CLI v2 Linux x86_64 installer was downloaded, extracted, installed with administrator privileges, and verified.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

3. Install kind and kubectl

kind was installed for running Kubernetes clusters in Docker. kubectl was then installed and verified. The captured environment reports kubectl client version v1.33.4.

```
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.23.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
kind --version
kubectl version --client
```

4. Install and verify helper tools

OpenSSL was checked to ensure cryptographic tooling was available. Trivy was run in a Docker container to scan the Alpine image; the captured scan reported zero vulnerabilities.

```
openssl version
docker run --rm aquasec/trivy image alpine
```

5. Create and validate the Kubernetes cluster

A local kind cluster named ccse was created. The command created the control-plane node and set the active Kubernetes context to kind-ccse. Node status and cluster connectivity were then checked.

```
kind create cluster --name ccse
kubectl get nodes
kubectl cluster-info --context kind-ccse
```

6. Start LocalStack

LocalStack was started as a detached Docker container with its edge service published on port 4566. The running container was verified with docker ps.

```
docker run -d --name localstack -p 4566:4566 localstack/localstack
docker ps
```

7. Configure AWS CLI for LocalStack

The AWS CLI was configured with LocalStack test credentials and a local endpoint. An STS get-caller-identity request returned account ID 000000000000, confirming AWS CLI communication with LocalStack.

```
aws configure set aws_access_key_id test
aws configure set aws_secret_access_key test
aws configure set region us-east-1
EP='--endpoint-url=http://localhost:4566'
aws $EP sts get-caller-identity
```

Commands Used

The commands used are listed in the implementation steps above: Docker verification and LocalStack commands; AWS CLI v2 installation and LocalStack configuration; kind and kubectl installation and cluster commands; and OpenSSL and Trivy verification commands.

Challenges Encountered

The Docker convenience-install script attempted to use a Docker Debian repository for kali-rolling, which did not provide a valid Release file. The terminal also showed hostname-resolution errors for the local hostname during that attempt. This was treated as an installation-method issue; Docker was subsequently verified successfully with docker run hello-world.

Lessons Learned

A successful container run is a stronger Docker check than only confirming that the command is installed.
Package repositories must match the Linux distribution and release.
LocalStack supports AWS CLI practice without real cloud credentials or charges.
kind provides a practical local Kubernetes cluster by running nodes as Docker containers.
Container-image scanning should be part of a basic cloud-development workflow.

References

IKB42603_Lab0_Environment_Setup_Cheatsheet.pdf (provided lab guide).
Docker documentation: https://docs.docker.com/.
AWS CLI v2 installation guide: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html.
kind documentation: https://kind.sigs.k8s.io/.
Kubernetes kubectl documentation: https://kubernetes.io/docs/reference/kubectl/.
LocalStack documentation: https://docs.localstack.cloud/.
Trivy documentation: https://trivy.dev/latest/.

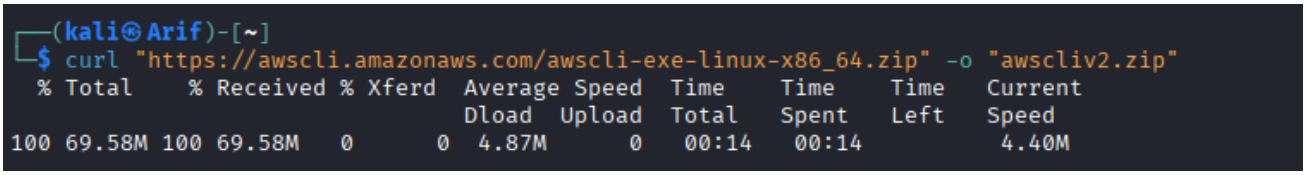
Screenshots

The following screenshots are the evidence captured during the environment setup.

Docker hello-world verification

```
┌──(kali㉿Arif)-[~]
└─$ docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

AWS CLI installer download

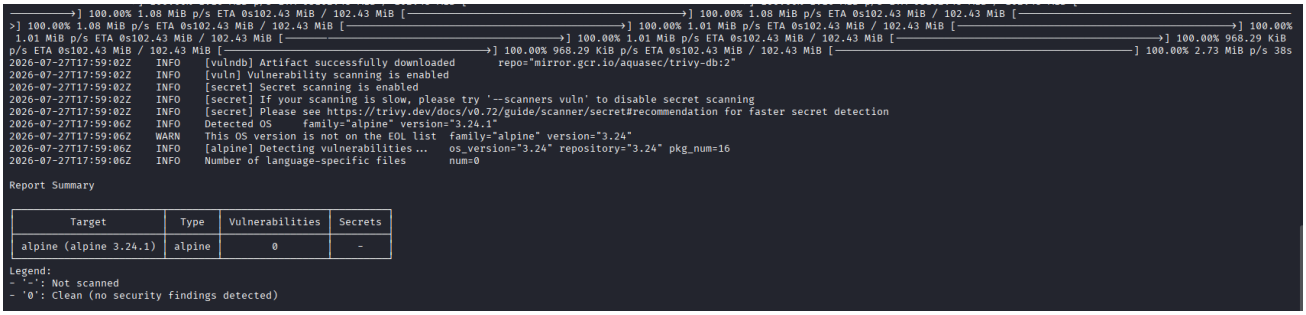


kind download

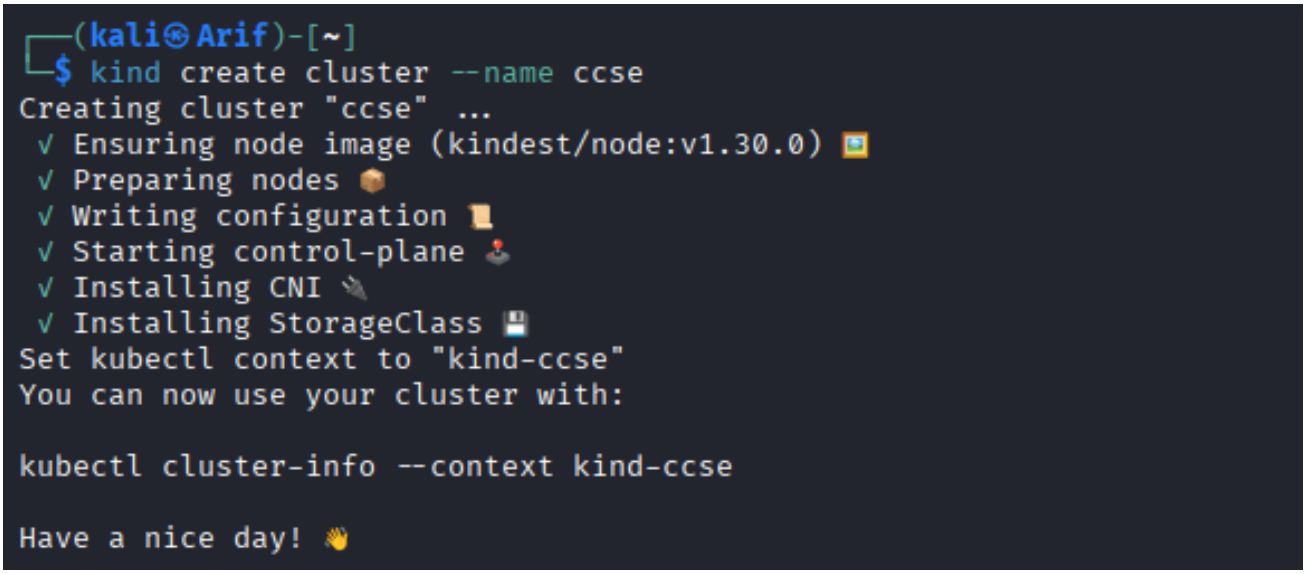
```
┌──(kali㉿Arif)-[~]
└─$ curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.23.0/kind-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100     97 100     97    0     0    249      0                              0
  0      0   0      0    0     0      0      0                              0
100  6.23M 100  6.23M    0     0  3.17M      0   00:01   00:01          6.11M
```

kubectl client verification

Trivy scan of Alpine image



kind cluster creation



LocalStack image download and container start

```
┌──(kali㉿Arif)-[~]
└─$ docker run -d --name localstack -p 4566:4566 localstack/localstack
Unable to find image 'localstack/localstack:latest' locally
latest: Pulling from localstack/localstack
0b83a2e2494a: Pull complete
0132b37780a2: Pull complete
0b986499c20f: Pull complete
9a40bf2386c3: Pull complete
4f4fb700ef54: Pull complete
a72524a5b829: Pull complete
0eb7a2a2c49e: Pull complete
1e7dd6779d16: Pull complete
d9e9c6298d04: Pull complete
03f05ee11ce5: Pull complete
f68fd5265233: Pull complete
fe6014eb39fe: Pull complete
9a064ef7c926: Pull complete
69c1481b64e8: Pull complete
7be01e8f61bc: Pull complete
1cd161bf20ec: Pull complete
86c83936dd33: Pull complete
22261b5c0eac: Pull complete
590247245e75: Pull complete
a66844e1ed68: Pull complete
Digest: sha256:2a81e5da4c32bb53e8d86e92050a12937f9be1915c5a4afad0931f75c112fc7e
Status: Downloaded newer image for localstack/localstack:latest
3b3820d86423dafa00ecd749591dad91ae4bb1718405c200f3e87da70cc39923
```

LocalStack container verification

```
┌──(kali㉿Arif)-[~]
└─$ docker run -d --name localstack -p 4566:4566 -e LOCALSTACK_AUTH_TOKEN="ls-zoTUbeNA-xon
E-1944-9184-ciDOMaPef45c" localstack/localstack
9be145a33d600f2fe8f1e39f73f898f1106abf5e9c4eeb3e2f75b88f99db3a83

┌──(kali㉿Arif)-[~]
└─$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS
                     PORTS
                    NAMES
9be145a33d60   localstack/localstack   "docker-entrypoint.sh"   4 seconds ago   Up 4 secon
ds (health: starting)   53/tcp, 443/tcp, 4510-4559/tcp, 5678/tcp, 0.0.0.0:4566→4566/tcp,
[::]:4566→4566/tcp   localstack
```

AWS CLI connected to LocalStack

```
┌──(kali㉿Arif)-[~]
└─$ aws configure set aws_access_key_id test

┌──(kali㉿Arif)-[~]
└─$ aws configure set aws_secret_access_key test
aws configure set region us-east-1

┌──(kali㉿Arif)-[~]
└─$ EP='--endpoint-url=http://localhost:4566'

┌──(kali㉿Arif)-[~]
└─$ aws $EP sts get-caller-identity
{
    "UserId": "000000000000",
    "Account": "000000000000",
    "Arn": "arn:aws:iam::000000000000:root"
}
```